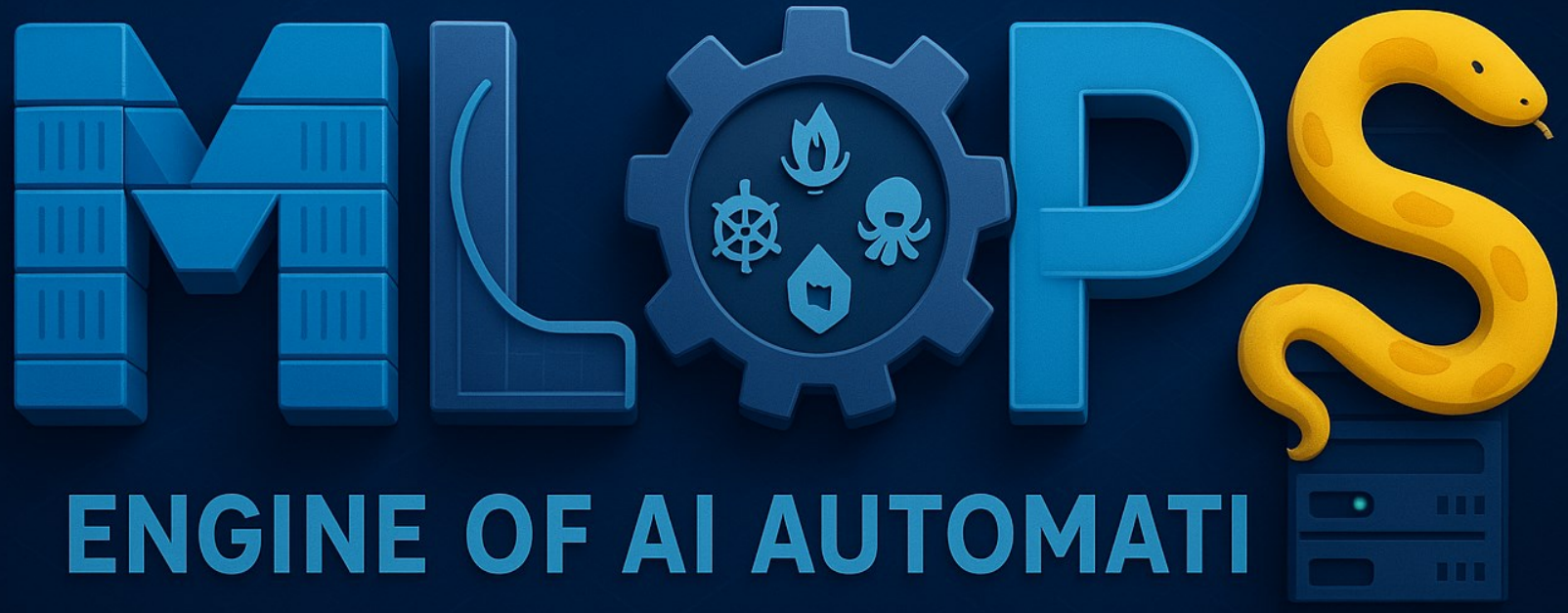
Основы MLOPS

Уральский федеральный университет
имени первого Президента России Б.Н.Ельцина



Антон Александрович Кошелев

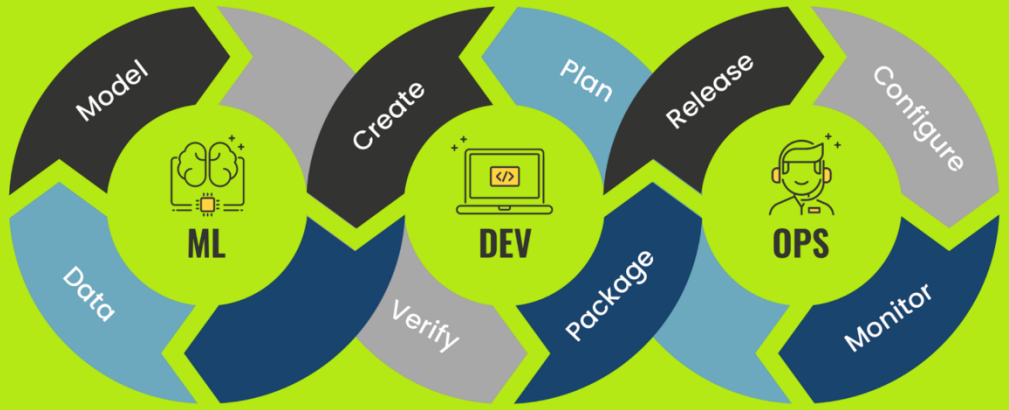
План занятия

1. Основы качественного кода в ML-проектах (Codestyle)
2. GitLab и Gitlab Flow в MLOps
3. Изоляция и управление средами и инструменты для управления зависимостями
4. Интеграция с CI/CD и автоматизация
5. Лучшие практики

MLOps (Machine Learning Operations)

это совокупность практик, объединяющих машинное обучение и DevOps для создания, тестирования, деплоя и сопровождения моделей машинного обучения (ML).

Основная цель MLOps — автоматизация и повышение эффективности всех этапов жизненного цикла ML-модели.



Без MLOps:

- Разработка модели на ноутбуке дата-сайентиста.
- Ручное тестирование и корректировка кода.
- Ручное внедрение модели в продакшн.
- Трудности с обновлением модели при изменении данных.

С MLOps:

- Код и данные версионируются в GitLab, обеспечивая отслеживаемость изменений.
- Автоматизированные тесты и CI/CD-пайплайны проверяют код на ошибки и автоматически деплоят обновления модели.
- Модель легко обновляется при изменении данных благодаря встроенным инструментам мониторинга и версионирования моделей.

Разработка модели

➔ Дата-сайентисты создают **модель**, используя языки программирования (чаще всего Python) и различные библиотеки для ML (например, Scikit-learn)

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

Тестирование

➔ Модели и их код проверяются на наличие ошибок и проблем производительности с помощью **автоматизированных** тестов.

```python
def test_model_accuracy():
    model = LinearRegression()
    model.fit(X_train, y_train)
    accuracy = model.score(X_test, y_test)
    assert accuracy > 0.8
```

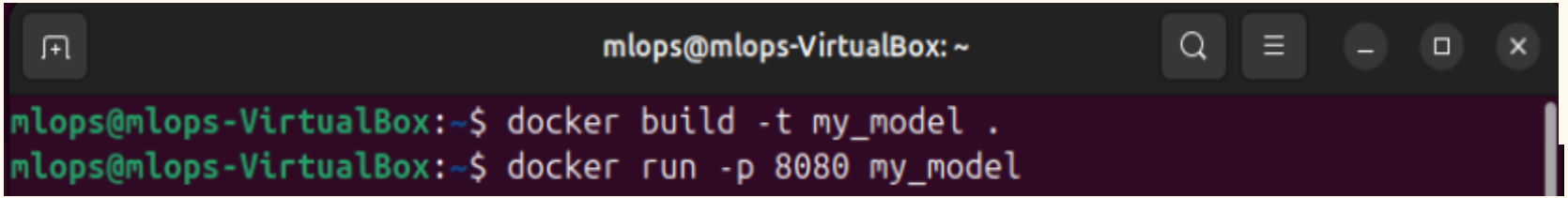
CI/CD (Continuous Integration/Continuous Deployment)

➔ Автоматизация процесса интеграции новых версий кода и моделей, а также их деплоя в продакшн.



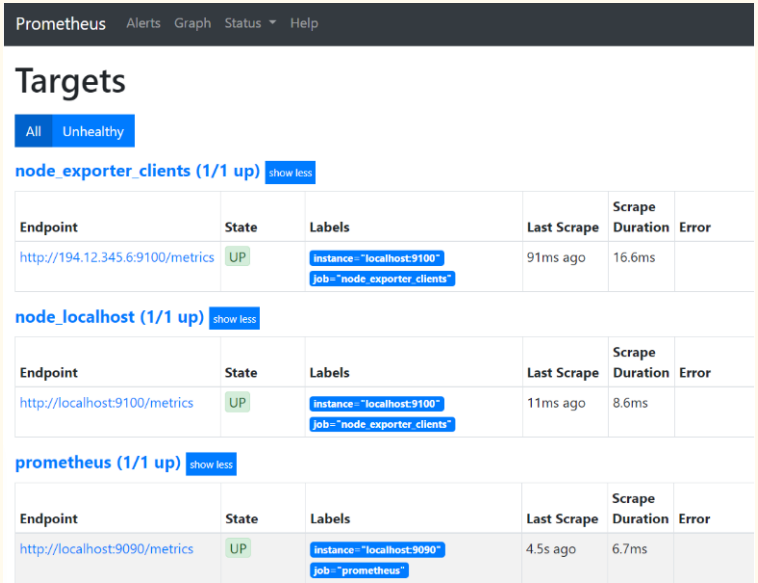
Внедрение и деплой

➔ Модель внедряется в продакшн-среду, где используется для прогнозирования на новых данных.



Мониторинг и сопровождение

- ➔ После деплоя важно отслеживать, как модель работает в реальной среде, следить за её точностью и своевременно обновлять её при необходимости.
- ➔ Модель отправляет метрики о точности и времени ответа, которые затем отображаются на дашбордах.



Codestyle

Для проектов машинного обучения (ML) важен не только результат, но и сам процесс разработки.

Написание чистого и стандартизированного кода — залог успешного сотрудничества в командах и долгосрочной поддержки проекта.

Дополнительно: [habr-статья]



Проблемы, которые решает codestyle

- ➔ Легкость чтения и понимания кода новыми членами команды.
- ➔ Упрощение процесса отладки и исправления ошибок.
- ➔ Стандартизированные способы написания кода помогают избежать ошибок при слиянии различных частей проекта.
- ➔ Автоматизация проверки качества кода с помощью линтеров и форматтеров позволяет уменьшить количество ручных ошибок.



Принципы качественного кода:

- **Читабельность**. Код должен быть понятным даже тем, кто его не писал.

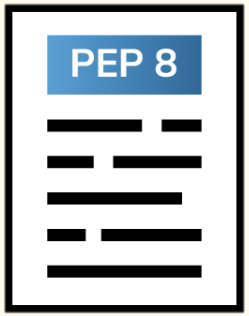
```
def calculate_accuracy(y_true, y_pred):
    correct_predictions = sum(y_true == y_pred)
    total_predictions = len(y_true)
    accuracy = correct_predictions / total_predictions
    return accuracy
```

- **Поддерживаемость**. Код должен быть легко изменяемым и расширяемым.
- **Тестируемость**. Код должен быть структурирован так, чтобы его можно было легко протестировать.

Краткий обзор PEP8 как стандарта кодирования в Python

PEP8 — это официальный стиль кодирования Python.





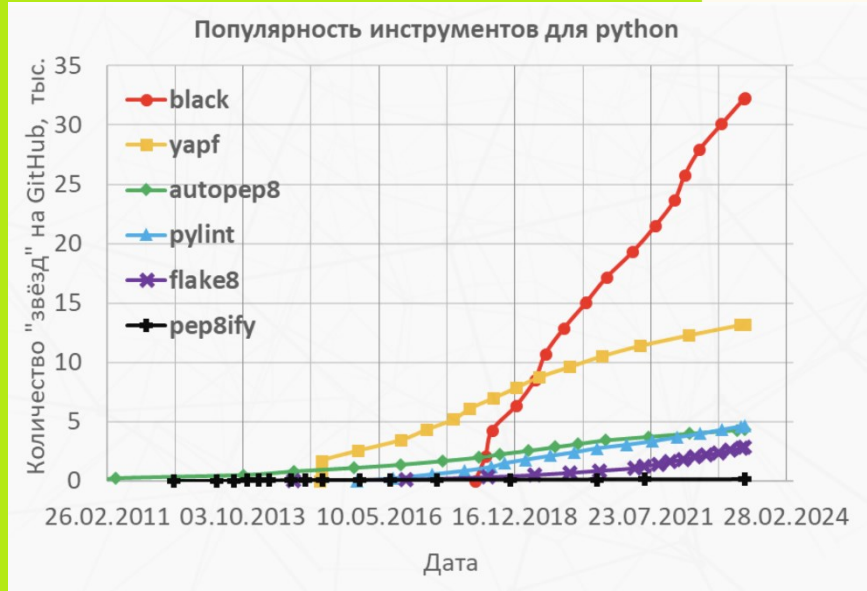
Основные моменты:

- ➔ Использовать **4 пробела** для отступов.
- ➔ Давать **осмысленные имена** переменным и функциям.
- ➔ Оставлять строки не длиннее **79 символов**.
- ➔ Использовать **пробелы** вокруг операторов (например, **a + b**).

Code formatters

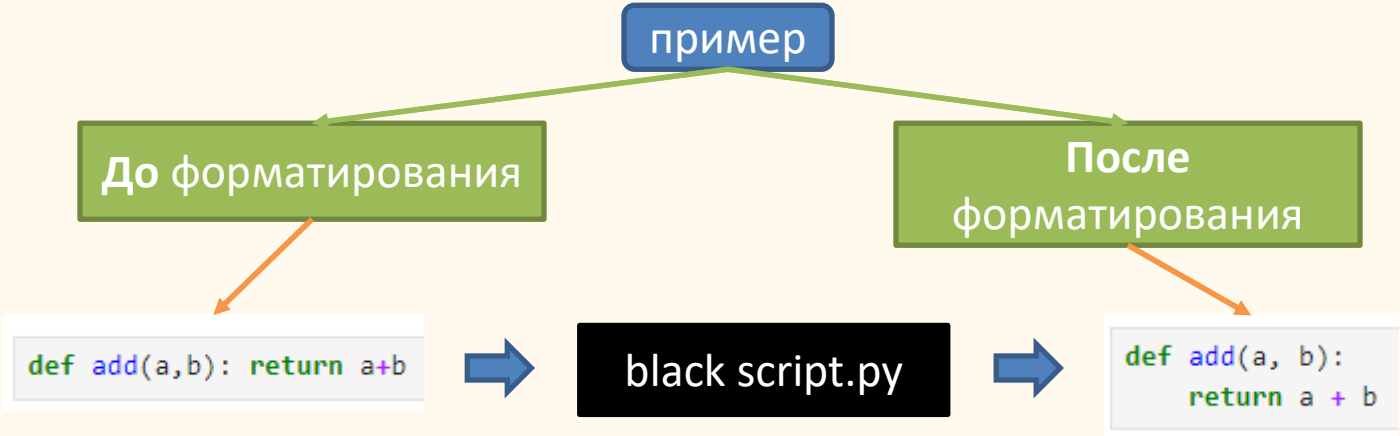
это инструменты, которые автоматически форматируют ваш код в соответствии с установленными стандартами.

Они избавляют от необходимости вручную исправлять пробелы, отступы и оформление.



Code formatters Black

Black — это популярный форматтер для Python, который автоматически форматирует код в соответствии с PEP8 и делает его однородным. Black применяет строгий подход: не требует настроек и автоматически форматирует код по своим правилам.



Code linters Flake8

Flake8 — это линтер для Python, который проверяет код на наличие синтаксических ошибок, стиля и потенциальных багов.

Flake8 объединяет несколько проверок:

- ➔ **PyFlakes** — проверка на ошибки и баги в коде.
- ➔ **pep8** — проверка на соответствие PEP8.
- ➔ **McCabe** — проверка на сложность кода.

Демонстрация Flake8

Пример некорректного кода

```
def add(a,b):
    return a +b
```

Запустить flake8 можно при помощи команды **flake8 script.py**

Вывод:

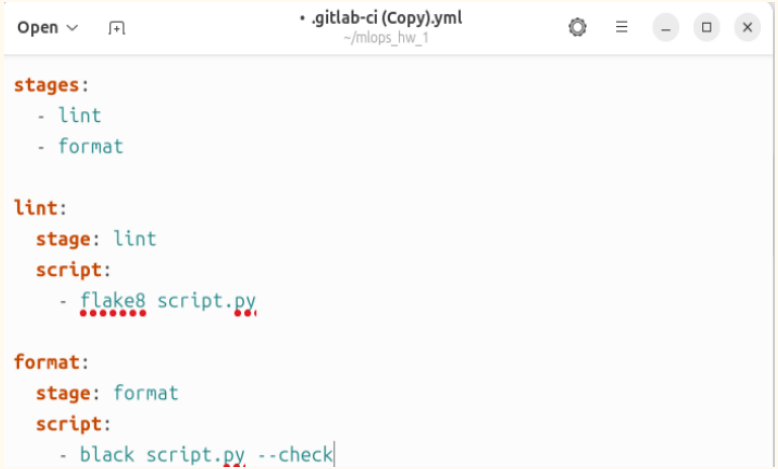
```
script.py:1:10: E231 missing whitespace after ','
script.py:2:12: E231 missing whitespace after '+'
```

Flake8 выведет список всех ошибок и предупреждений, таких как несоответствия PEP8 или возможные баги.

Внедрение линтеров и форматтеров в pipeline для автоматизации контроля качества кода

Чтобы автоматизировать процесс проверки качества кода, форматтеры и линтеры можно встроить в **CI/CD пайплайн**, используя **GitLab CI**.

Пример пайплайна в GitLab CI:



Gitlab в MLOps

это мощный инструмент для управления проектами, контроля версий и автоматизации процессов разработки, что делает его ключевым элементом в MLOps.



GitLab

Зачем нужен GitLab для MLOps проектов

- ➔ Контроль версий.
- ➔ Управление проектами.
- ➔ Инструмент CI/CD в GitLab.

Пример того как выглядит CI/CD pipeline в Gitlab

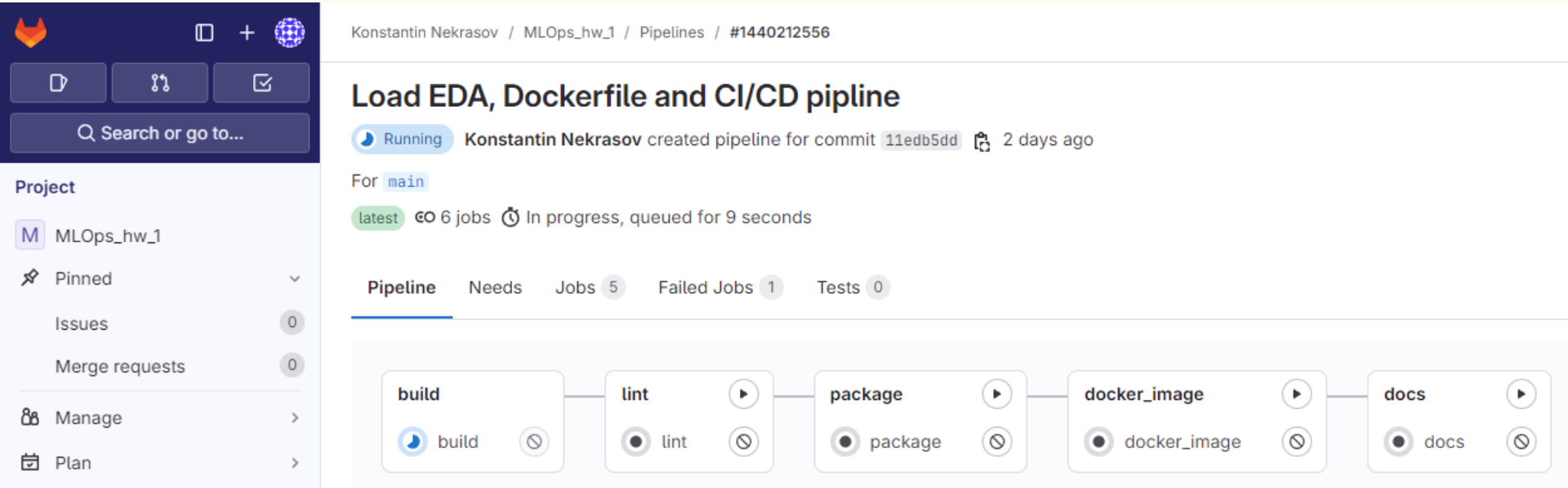
Name	Last commit	Last update
.ipynb_checkpoints	Load EDA, Dockerfile and CI/CD pipline	2 days ago
__pycache__	Load EDA, Dockerfile and CI/CD pipline	2 days ago

0 Tags

112.4 MiB Project Storage

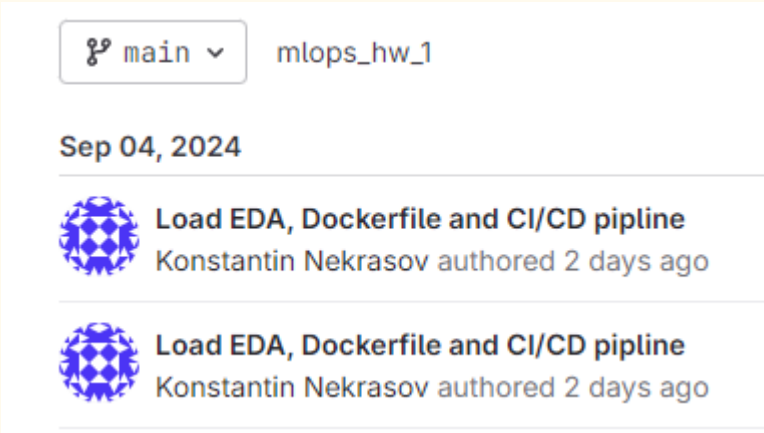
README

CI/CD configuration



Управление версиями кода и моделей

- ➔ Версионность кода в MLOps
- ➔ Отслеживание изменений в данных и моделях



Пример: настройка простого GitLab репозитория для ML-проекта

➔ Создание репозитория:

- Войдите в GitLab и создайте новый репозиторий (**project**).
- Настройте .gitignore файл для исключения временных файлов и артефактов.



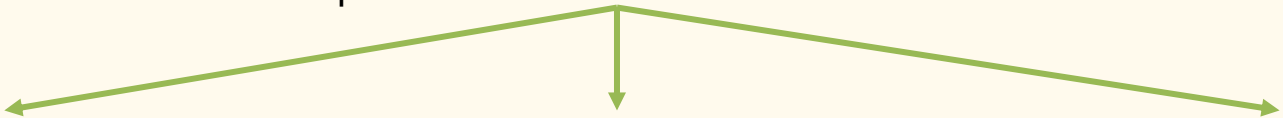
➔ Создание базового **CI/CD пайплайна** .gitlab-ci.yml (пример на слайде 24)

GitLab Flow:
Стратегии управления ветками

это подход к управлению ветками, который помогает организовать эффективную работу над проектом, поддерживая версионирование, код-ревью и деплой в продакшн.

Основные стратегии ветвления

Существует 3 основные стратегии ветвления



➔ **Main/Production** ветка

➤ Эта ветка (часто называется **main** или **master**) хранит стабильную и рабочую версию кода, готовую к деплою.

➤ В неё сливаются только проверенные и протестированные изменения.

➔ **Feature** ветки

➤ Каждая новая функциональность, улучшение или исправление ошибки разрабатываются в отдельной ветке, которая создается от **main.**

➤ Это позволяет работать над функциями параллельно, не нарушая стабильности основной ветки.

➔ **Merge Requests** и **код-ревью**

➤ После завершения работы над feature-веткой разработчик создает **Merge Request (MR)** для слияния изменений в основную ветку.

➤ В процессе слияния другие члены команды могут просмотреть изменения, внести правки и провести код-ревью перед окончательным слиянием.

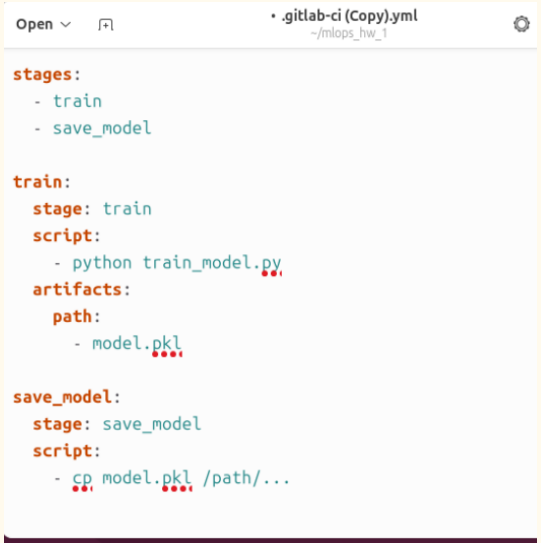
Пример: как правильно организовать ветвление в GitLab

- ➔ Создание новой feature ветки при помощи команды**: git checkout -b new-algorithm**
- ➔ Создание **Merge Request**. В GitLab создается Merge Request для **слияния** вашей **ветки в основную**.

Применение GitLab Flow для управления данными и моделями

- **Feature ветки для данных**. Если данные меняются, можно создать отдельную ветку для их обработки, протестировать работу с новой версией данных, а затем объединить изменения.

- **Модели как артефакты**. Модели можно версионировать как артефакты в CI/CD пайплайне. При успешном обучении новая версия модели сохраняется как артефакт и может быть использована для деплоя.



Заключение

Мы изучили значение чистого кода в ML-проектах и использование инструментария для его автоматизации (Black, Flake8). Рассмотрели роль GitLab в управлении версиями, проектами и CI/CD. Также освоили GitLab Flow и стратегии ветвления для эффективной разработки и управления ML-моделями.

Изоляция и управление средами

Уральский федеральный университет

Среда разработки

это совокупность настроек и зависимостей (библиотек, версий Python и т.д.), необходимых для запуска вашего приложения или модели.

Почему нужна изоляция среды для проекта?

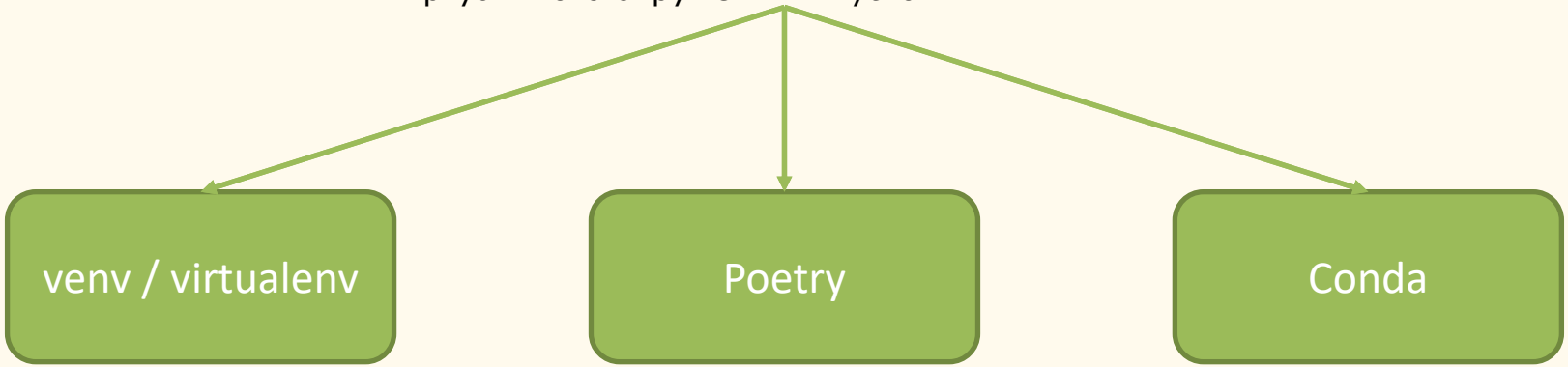
- **Конфликты версий**. Разные проекты могут требовать разные версии одних и тех же библиотек.
- **Репродуктивность.** Гарантия того, что код будет работать одинаково на разных системах.
- **Безопасность.** Изоляция помогает ограничить воздействие потенциально вредоносного кода.

Виртуальные окружения

позволяют создать изолированную среду для Python-проектов, где устанавливаются зависимости, не влияющие на систему в целом.



Рассмотрим **3 инструмента** для создания виртуального окружения в Python



venv

1. Встроенный инструмент Python для создания виртуальных окружений.

2. Простой в использовании и интегрируется в любую версию Python.

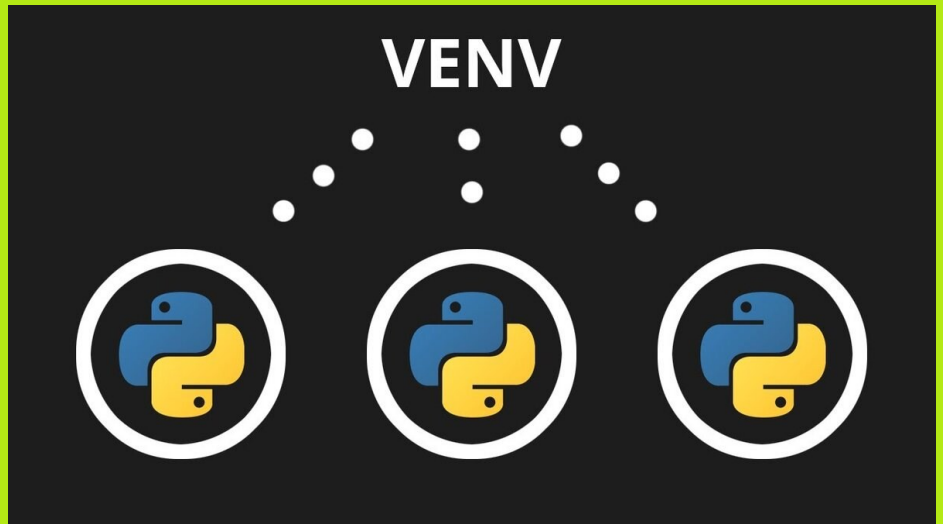
virtualenv

1. Больше возможностей (например, поддержка различных интерпретаторов Python).

2. Быстрая активация и настройка виртуальных окружений.

venv

это встроенный в Python модуль для создания виртуальных окружений



virtualenv

это сторонний модуль для создания виртуальных окружений в Python, который предлагает дополнительные возможности по сравнению со встроенным модулем.

virtualenv доп. возможности

- ➔ Совместимость с разными версиями Python
- ➔ Быстрая скорость создания окружений
- ➔ Дополнительные опции и гибкость
- ➔ Копирование вместо символических ссылок (virtualenv --copies myenv)
- ➔ Поддержка плагинов и расширений:

- virtualenvwrapper

mkvirtualenv myenv
workon myenv
rmvirtualenv myenv

- virtualenv-clone

virtualenv-clone old_env new_env

virtualenv доп. возможности

- ➔ Создание переносимых окружений: **virtualenv --relocatable myenv**
- ➔ Совместимость с более старыми проектами и инструментами
- ➔ Более тонкий контроль над окружением
- ➔ Поддержка оболочек Windows
- ➔ Совместимость с альтернативными реализациями Python
- ➔ Интеграция с инструментами автоматизации

venv / virtualenv

- ➔ Создание виртуального окружения: **python3 -m venv (/virtualenv) myenv**
- ➔ Активация виртуального окружения: **source myenv/bin/activate**
- ➔ Деактивация виртуального окружения: **deactivate**

Poetry

современный инструмент для управления зависимостями и упаковки Python-проектов.



Poetry: особенности

- ➔ Управление зависимостями и виртуальными окружениями в одном инструменте.
- ➔ Автоматическое разрешение конфликтов версий.
- ➔ Поддержка публикации пакетов.



Создание и настройка проекта с Poetry

- ➔ Установка Poetry: curl -sSL https://install.python-poetry.org | python3 -
- ➔ Создание нового проекта: **poetry new first**

```
poetry new first

Created package first in first

cd first  # Переходим в созданную директорию
tree .

.
├── first
│   └── __init__.py
├── pyproject.toml
├── README.md
└── tests
    └── __init__.py
```

Управление зависимостями

- ➔ Добавление зависимости: **poetry add numpy**
- ➔ Добавление зависимости для разработки: **poetry add --dev pytest**
- ➔ Команду обновления окружения можно вызвать: **poetry install**

```
poetry install

Updating dependencies
Resolving dependencies...

Writing lock file

…

Installing the current project: first (0.1.0)
```

Управление виртуальными окружениями

- ➔ Активация виртуального окружения: **poetry shell**
- ➔ Выполнение команды в окружении: **poetry run python script.py**
- ➔ Деактивация виртуального окружения: **exit**

Плюсы Poetry

1. Интегрированное управление зависимостями и окружениями.

2. Простая публикация пакетов.

3. Автоматическое разрешение зависимостей.

Минусы Poetry

1. Может быть избыточным для простых проектов.

2. Требует изучения нового инструмента ☺

Conda

это кроссплатформенный менеджер пакетов и сред, не зависящий от языка программирования.



Особенности Conda

- ➔ Поддержка бинарных пакетов (ускоряет установку и исключает необходимость компиляции).
- ➔ Поддержка Python, R и других языков.
- ➔ Возможность создания и управления несколькими изолированными средами для разных проектов.
- ➔ Удобен для работы с большими проектами, особенно для науки о данных (Anaconda).

Установка Conda

- ➔ **Anaconda.** Полная дистрибуция с множеством пакетов.
- ➔ **Miniconda.** Минимальная установка Conda.

1. Скачивание Miniconda: wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
2. Установка: bash Miniconda3-latest-Linux-x86_64.sh

Управление BO с Conda

- ➔ Создание окружения: **conda create -n myenv python=3.9**
- ➔ Активация окружения: **conda activate myenv**
- ➔ Удаление окружения: **conda remove -n myenv --all**

Управление пакетами с Conda

- ➔ Установка пакета из каналов Conda: **conda install numpy**
- ➔ Установка пакета из канала conda-forge: **conda install -c conda-forge package_name**
- ➔ Установка пакета с помощью pip внутри Conda-окружения: **pip install package_name**

Плюсы Conda

1. Управление как Python, так и системными зависимостями.

2. Упрощенная установка сложных пакетов (например, TensorFlow, PyTorch).

3. Поддержка нескольких языков.

Минусы Conda

1. Большой размер установки.

2. Возможные конфликты между pip и Conda-пакетами.

Сравнение venv, Poetry и Conda

Особенность	**venv**	**Poetry**	**Conda**
Управление зависимостями	Да	Да	Да
Управление окружениями	Нет	Да	Да
Поддержка системных пакетов	Нет	Нет	Да
Автоматическое разрешение зависимостей	Ограничено	Да	Да
Размер установки	Маленький	Маленький	Большой
Простота использования	Простая	Средняя	Средняя

Какие еще есть инструменты?

Помимо **venv/virtualenv, Poetry, Conda**, есть также инструменты:

- **hatch**
- **pixi**
- **rye(uv)**
- **pdm**



Общее сравнение инструментов

Категория/Инструмент	pipenv	poetry	conda	hatch	pixi	rye(uv)	pdm
Управление версией Python	✗	✗	✓	✗	✓	✓	✓
Виртуальные окружения	✓	✓	✓	✓	✓	✓	✓
Управление пакетами	✓	✓	✗	✓	✓	✓	✓
Управление научными пакетами	✗	✗	✓	✗	✓	✗	✗
Воспроизводимые зависимости	✓	✓	✗	✗	✓	✓	✓
Сборка и публикация пакетов	✗	✓	✗	✓	✗	✓	✓
Сборка и публикация научных пакетов	✗	✗	✓	✗	✗	✗	✗
Единый файл мета данный	✗	✗	✓	✓	✗	✓	✓
Автоматизация выполнения команд	✗	✗	✗	✓	✓	✓	✓
Шаблонизация структуры проекта	✗	✗	✗	✗	✗	✗	✓

Рекомендации при работе с ВО

- ➔ Один проект — **одно окружение**. Это упрощает управление зависимостями.
- ➔ Не храните окружения в системе контроля версий. Добавьте папку окружения в **.gitignore**
- ➔ Используйте понятные имена окружений. Например, **env, venv** или имя проекта.

Интеграция с CI/CD и автоматизация

Уральский федеральный университет

Установка зависимостей

При каждом запуске CI/CD конвейера, особенно в больших проектах с множеством зависимостей, установка пакетов может занимать значительное время. Кэширование зависимостей позволяет сохранить ранее установленные пакеты и использовать их в последующих сборках, значительно ускоряя процесс.

Установка зависимостей в конвейерах

Примеры кэширования в GitLab CI/CD

gitlab-ci.yml

: > Users > NeKonn > Downloads > ! gitlab-ci.yml

```yaml
cache:
  key: "$CI_COMMIT_REF_SLUG"
  paths:
    - .cache/pip

stages:
  - build

build_job:
  stage: build
  script:
    - pip install -r requirements.txt --cache-dir .cache/pip
```

Рекомендации по кэшированию

- Фиксируйте версии зависимостей. Используйте **requirements.txt** или **poetry.lock**, чтобы гарантировать, что кэш соответствует необходимым версиям.

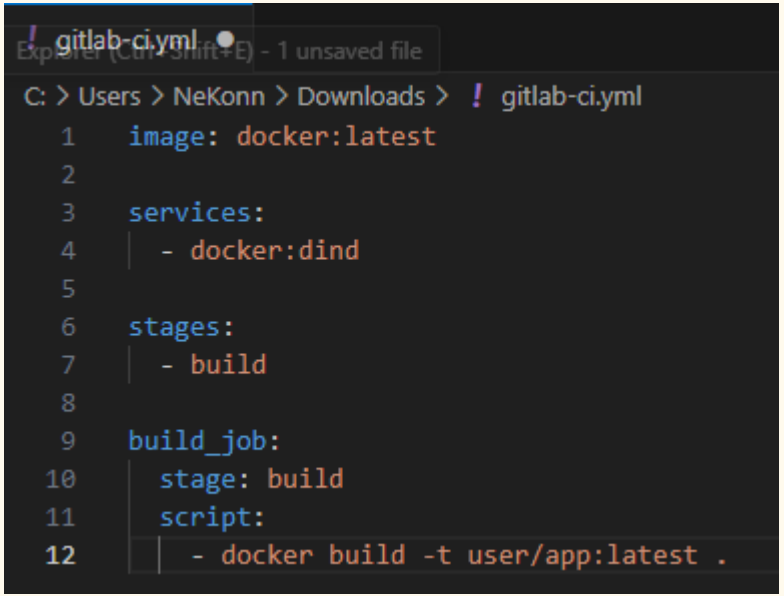
- Обновляйте кэш при изменении зависимостей. Используйте хэш файла зависимостей в качестве ключа кэша.

Использовать контейнер для консистентности окружений

Контейнеризация позволяет создать единое окружение разработки и производства, обеспечивая консистентность и воспроизводимость. Контейнеры включают в себя все необходимые зависимости и конфигурации, что упрощает развёртывание и тестирование.

Docker в CI/CD конвейере

Пример использования GitLab CI/CD с Docker



Управление секретами и переменными окружения

В процессе разработки и особенно при развёртывании приложений важно безопасно управлять конфиденциальной информацией, такой как API-ключи, пароли и другие секреты.

Безопасное хранение секретов

- Добавьте .env в .gitignore, чтобы предотвратить его коммит.
- Используйте .env.example для указания необходимых переменных без значений.

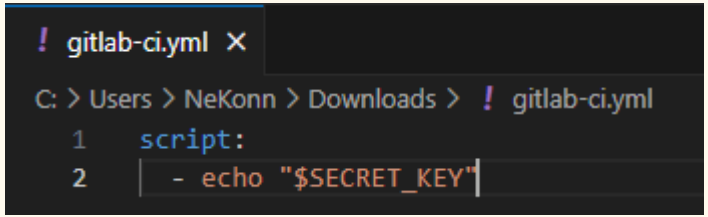


Передача переменных окружения

Использование переменных окружения позволяет конфигурировать приложение без изменения кода.

Шифрование переменных в CI/CD системах

Пример использования GitLab CI/CD

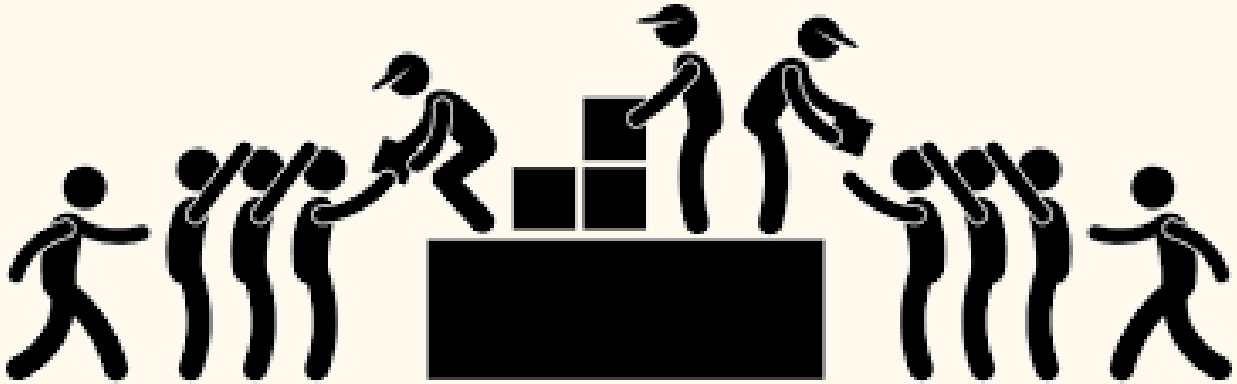


Где SECRET_KEY — это защищённая переменная, заданная в настройках проекта.

Лучшие практики

Уральский федеральный университет

- Управление зависимостями в больших командах
- Миграция между инструментами



Конфликты версий при обновлении

Проблема:
- При обновлении зависимостей могут возникать конфликты версий, приводящие к поломке приложения или несовместимостям.

Решение:
- Создание изолированных окружений для тестирования обновлений
- Постепенное обновление и проверка совместимости
- Использование инструментов для разрешения конфликтов

Проблемы с совместимостью платформ

Проблема:
- Приложение работает на одной операционной системе, но не работает на другой из-за различий в зависимостях или окружении.

Решение:
- Тестирование на разных операционных системах
- Использование контейнеризации

Уральский федеральный университет

БЛАГОДАРЮ ЗА ВНИМАНИЕ

Задание на практику

Уральский федеральный университет

1. Сдать на hh.ru тесты по:
    1. Машинное обучение
    2. Python. Теория и практика
    3. SQL. Теория и практика
    4. Docker, Git, Linux, ООП
2. Рефакторинг кода ML-проекта под production-стандарты.
3. Настройка pre-commit, poetry, линтеров.
4. Интеграция в Git-репозиторий команды.